C++ OOP LAB
VIVA QUESTION BANK

Detailed questions and easy-to-say answers from the supplied programs

11 topic groups | 25 unique programs | code-specific follow-ups | dry runs | edge cases | comparison questions | mock viva

How to practice: Start with the quick questions. Then practise the expanded answers as if explaining at the board: define the idea, trace the supplied code, state an edge case, and mention the trade-off. Questions tagged **[General knowledge]** go beyond the source program.

Based on the supplied pcc.md transcription. Its code was not compiled or run. Programs repeated in the PDFs are not duplicated in the source notes.

Contents

Experiment 1 - Number Conversion and Student Structure 3

Experiment 2 - GCD/LCM and Shopping List Class 5

Experiment 3 - Function Overloading and Friend Function 7

Experiment 4 - Parameterized and Copy Constructors 9

Experiment 7 - Dynamic Memory Allocation 11

Experiment 8 - Text File Handling with Streams 13

Experiment 8B - Binary File I/O 15

Experiment 9 - Function Templates and Bubble Sort 17

Experiment 10 - Exception Handling 19

PDF Program 4(C) - Multiple Inheritance and Construction Order 21

PDF Program 5(A) - Hybrid and Virtual Inheritance 23

PDF Programs 5(B), 6(A), 6(B) - Runtime Polymorphism 25

Open-Ended 1 - Lambda with Reference Capture 27

Open-Ended 2 - std::vector Operations 29

The first part follows the source-file programs; the later sections cover additional PDF programs and open-ended experiments.

Experiment 1 - Number Conversion and Student Structure

Program focus: ex01a uses hex, oct, dec and showbase. ex01b groups college, department, student, three marks, percentage, and CGPA in a struct.

Quick viva questions

Q1. What does a C++ struct do?

Easy answer. It groups related data under one type. The Student structure stores strings, an array of three marks, percentage, and CGPA in one object.

Q2. How does number conversion work in ex01a?

Easy answer. The hex, oct, and dec stream manipulators choose the base used to print the same integer. showbase adds a conventional base prefix; the value itself does not change.

Q3. Do stream manipulators keep affecting later output?

Easy answer. The base setting remains until another base manipulator changes it. The code explicitly uses hex, then oct, then dec.

Q4. What does getline read here?

Easy answer. It reads a whole line, including spaces, so a college, department, or student name can contain multiple words.

Q5. How is totalMarks calculated?

Easy answer. The loop reads three subject marks and adds each to totalMarks.

Q6. What formulas produce percentage and CGPA?

Easy answer. percentage=(totalMarks*100)/300, assuming three subjects each have maximum 100. CGPA is then percentage/10, which is a simplified formula used by this program.

Q7. What input validation is missing?

Easy answer. It does not check that marks are between 0 and 100 or that input succeeded.

Q8. Dry run number conversion for 26.

Easy answer. Hexadecimal is 0x1a, octal is 032 with showbase, and decimal is 26.

Q9. when is a struct appropriate? [General knowledge]

Easy answer. It is useful when a group of related fields needs to travel together as a record; C++ structs can also have member functions and constructors.

Experiment 1 - Number Conversion and Student Structure - expanded answers

Practice question 1. Explain the number-conversion program.

Suggested spoken answer. The program reads one integer and prints that same value in three bases. The hex manipulator selects base 16, oct selects base 8, and dec returns to base 10. showbase asks the stream to display a prefix, so a value such as 26 appears as 0x1a, 032, and 26.

Practice question 2. Explain the Student structure and calculations.

Suggested spoken answer. Student is a user-defined record that keeps identity fields together with three subject marks and calculated results. The loop totals the marks; percentage assumes a maximum total of 300, and the code uses percentage divided by 10 as its CGPA rule. That rule is specific to this program and may not match every grading system.

Practice question 3. Why use getline for names?

Suggested spoken answer. Formatted extraction with >> stops at whitespace, but getline reads the complete line. Since this program uses getline for all three text fields before it starts numeric extraction, there is no leftover newline to discard at that point.

Experiment 2 - GCD/LCM and Shopping List Class

Program focus: ex02a uses long long, llabs, C++17 std::gcd, and a divide-before-multiply LCM formula. ex02b uses fixed arrays of 100 items and menu methods.

Quick viva questions

Q1. Define GCD and LCM.

Easy answer. GCD is the greatest positive integer dividing both values; LCM is the least positive common multiple, with the program defining LCM as zero if either input is zero.

Q2. Why use absolute values?

Easy answer. GCD and LCM are conventionally nonnegative. llabs converts signed inputs to magnitudes before std::gcd is called.

Q3. Why is the LCM formula (abs1/gcd)*abs2?

Easy answer. Dividing before multiplying avoids some unnecessary intermediate growth compared with abs1*abs2/gcd.

Q4. What happens for inputs 0 and 0?

Easy answer. std::gcd returns zero, so the conditional sets LCM to zero without dividing by zero.

Q5. What standard version provides std::gcd?

Easy answer. It is available from the numeric header starting in C++17.

Q6. What does count mean in Shopping?

Easy answer. It is the number of currently stored items and also the next array index used for insertion.

Q7. How does deleteItem work?

Easy answer. It searches for the first matching code, shifts later codes and prices left, reduces count, and returns.

Q8. What capacity issue is present?

Easy answer. The arrays have 100 elements, but addItem does not check whether count+n exceeds 100.

Q9. Compare totalPrice and deletion costs.

Easy answer. Both scan the current list in O(count); deletion also shifts elements after the match.

Q10. why can float money be problematic? [General knowledge]

Easy answer. Binary floating-point cannot represent many decimal fractions exactly; integer cents or a decimal type is safer for financial totals.

Experiment 2 - GCD/LCM and Shopping List Class - expanded answers

Practice question 1. Explain GCD and LCM in ex02a.

Suggested spoken answer. The program converts both inputs to magnitudes and calls std::gcd. If the GCD is zero, both inputs are zero and the code sets LCM to zero. Otherwise it calculates LCM by dividing one magnitude by the GCD first, then multiplying by the other magnitude.

Practice question 2. Dry run GCD and LCM for 12 and 18.

Suggested spoken answer. The common divisors include 1, 2, 3, and 6, so the GCD is 6. The LCM calculation is (12/6)*18=36, the smallest positive number divisible by both.

Practice question 3. Explain the shopping list storage and deletion.

Suggested spoken answer. The object stores item codes and prices in parallel arrays, while count marks how many entries are active. To delete a code, it finds the first match and shifts every later code and its corresponding price one position left. This keeps each code paired with its price, but insertion must check the fixed capacity before writing.

Experiment 3 - Function Overloading and Friend Function

Program focus: Area overloads take (float,float), (double,double), or (float). Maximum declares findMax as a friend, which reads private n1/n2.

Quick viva questions

Q1. What is function overloading?

Easy answer. Several functions share a name but have different parameter lists. The compiler selects an overload from the call's argument types and number of arguments.

Q2. Which area formulas does the source use?

Easy answer. Triangle: 0.5*base*height; rectangle: length*breadth; sphere: 4*3.14*radius^2.

Q3. Is the sphere formula volume or surface area?

Easy answer. It is surface area. Sphere volume would use (4/3)*pi*r^3.

Q4. Can return type alone distinguish overloads?

Easy answer. No. Overloads must differ in their parameter lists; changing only a return type does not make a distinct overload.

Q5. What is a friend function?

Easy answer. A non-member function granted access to a class's private/protected members.

Q6. Is findMax a member of Maximum?

Easy answer. No. It is a regular free function declared with friend inside the class.

Q7. Why does findMax take Maximum by value here?

Easy answer. It receives a copy of the object. A const reference could avoid that copy for a larger class.

Q8. What happens if n1 equals n2?

Easy answer. The else branch prints n2, which is the same maximum value.

Q9. What common overload issue should be watched for?

Easy answer. Calls with implicit conversions can be ambiguous if multiple overloads are equally viable.

Q10. static vs dynamic polymorphism? [General knowledge]

Easy answer. Overloading is resolved at compile time; virtual-function dispatch is resolved at run time.

Experiment 3 - Function Overloading and Friend Function - expanded answers

Practice question 1. Explain function overloading with the Area class.

Suggested spoken answer. The class has three functions named area, but their parameter lists differ. Two float arguments select the triangle overload, two double arguments select the rectangle overload, and one float selects the sphere overload. The compiler chooses the matching function before the program runs.

Practice question 2. Walk through the area formulas and identify a common viva correction.

Suggested spoken answer. The triangle area is one-half times base times height, and rectangle area is length times breadth. The sphere formula 4*pi*r-squared is the surface area of a sphere, not its volume; volume would use four-thirds pi r-cubed.

Practice question 3. Explain why findMax can read private fields.

Suggested spoken answer. Normally, a free function cannot access private members. Maximum declares findMax as a friend, granting that specific function access. It remains a non-member function and is called as findMax(m), not m.findMax().

Experiment 4 - Parameterized and Copy Constructors

Program focus: Student and Employee examples initialize fields from arguments; copy constructors accept const references and copy each field.

Quick viva questions

Q1. What is a constructor?

Easy answer. A special member function with the class name that initializes an object when it is created.

Q2. What makes a constructor parameterized?

Easy answer. It accepts arguments so the caller can provide initial member values.

Q3. What is a copy constructor?

Easy answer. A constructor that initializes a new object from an existing object of the same class, commonly with a const reference parameter.

Q4. What does employee e2=e1 do?

Easy answer. It copy-initializes e2 from e1 and invokes the copy constructor.

Q5. Why take the source object as const reference?

Easy answer. Reference avoids an unnecessary copy; const prevents the constructor from changing the source.

Q6. What fields are copied in these examples?

Easy answer. Every listed string and numeric field is explicitly assigned from the source object.

Q7. What happens if a class has no user-defined copy constructor?

Easy answer. The compiler generally provides a memberwise copy constructor, subject to the class's members and special-member rules.

Q8. How is copy construction different from assignment?

Easy answer. Construction creates a new object from another; assignment replaces the state of an already existing object.

Q9. What is the risk with raw owning pointers and default copying?

Easy answer. A shallow copy can make two objects own the same memory, causing double deletion; resource-owning types need correct copy/move behavior.

Q10. what is the Rule of Three? [General knowledge]

Easy answer. If a class needs a custom destructor, copy constructor, or copy assignment operator, it often needs all three.

Experiment 4 - Parameterized and Copy Constructors - expanded answers

Practice question 1. Explain parameterized constructors.

Suggested spoken answer. A parameterized constructor receives values at object creation and initializes the object's fields. In the Student example the caller supplies name, roll number, ID, and address; the Employee example similarly supplies employee information.

Practice question 2. Explain the copy-constructor example.

Suggested spoken answer. The program first creates e1 with a parameterized constructor. Then employee e2=e1 creates a new object by copying e1. The copy constructor accepts a const reference and copies each data member, so e2 has the same values but is a separate object.

Practice question 3. Compare copy construction and assignment.

Suggested spoken answer. Copy construction happens while a new object is being created, such as employee e2=e1. Copy assignment happens after both objects already exist, such as e2=e1;. This matters for classes that manage dynamic resources because copying must define who owns the resource.

Experiment 7 - Dynamic Memory Allocation

Program focus: The program validates n>0, allocates int[n] with new[], reads/displays values, and releases with delete[].

Quick viva questions

Q1. What is dynamic memory allocation?

Easy answer. It requests storage while the program is running, based on a run-time size.

Q2. What does new int[n] return?

Easy answer. A pointer to the first element of a dynamically allocated array of n integers.

Q3. Why check n<=0?

Easy answer. It rejects an invalid array size before allocation.

Q4. Why use delete[] rather than delete?

Easy answer. The memory came from array new[], so array delete[] must be used to release it correctly.

Q5. What is a memory leak?

Easy answer. Allocated memory that the program no longer references and never releases.

Q6. What is a dangling pointer?

Easy answer. A pointer that still refers to storage after it has been released.

Q7. What if allocation fails?

Easy answer. Ordinary throwing new reports failure by throwing std::bad_alloc; this code does not catch it.

Q8. What are safer modern C++ alternatives?

Easy answer. std::vector<int> or std::unique_ptr<int[]> manage lifetime automatically.

Q9. What does deleting the array before return accomplish?

Easy answer. It releases the dynamically allocated storage before main exits.

Q10. why is RAII useful? [General knowledge]

Easy answer. Resource Acquisition Is Initialization ties resource lifetime to object lifetime, making cleanup automatic.

Experiment 7 - Dynamic Memory Allocation - expanded answers

Practice question 1. Explain the allocation and cleanup sequence.

Suggested spoken answer. The program reads n and rejects nonpositive values. It allocates n integers with new[], fills and prints the array, then calls delete[] to return the storage. The brackets matter because the allocation created an array.

Practice question 2. What risks should you mention?

Suggested spoken answer. If allocation succeeds but a future code path exits without delete[], memory can leak. If memory is deleted and the pointer is still used, it dangles. The example has a simple path and deletes the array before returning, but it does not catch allocation failure.

Practice question 3. How would modern C++ simplify it?

Suggested spoken answer. A vector<int> grows dynamically and releases its memory automatically when it leaves scope. This follows RAII and removes the need to pair new[] with delete[] manually.

Experiment 8 - Text File Handling with Streams

Program focus: The collection writes/reads student.txt using streams directly and through a Student class; getline, >>, open checks, and close are shown.

Quick viva questions

Q1. What are ofstream and ifstream?

Easy answer. ofstream writes to files; ifstream reads from files. They are declared in the fstream header.

Q2. Why check if (!ofile) or if (!file)?

Easy answer. It detects failure to open the file, such as a permission or path problem.

Q3. Where is student.txt created?

Easy answer. Relative paths are resolved from the program's current working directory.

Q4. What happens when ofstream opens an existing file by default?

Easy answer. It normally truncates the file contents; use append mode if you want to add instead.

Q5. Why use getline for a name or address?

Easy answer. It reads spaces up to the newline, unlike formatted extraction with >> which stops at whitespace.

Q6. What does cin.ignore(numeric_limits<streamsize>::max(),'\n') do?

Easy answer. It discards leftover input through the next newline before getline reads the address.

Q7. How do the two text-file programs differ?

Easy answer. The first writes separate raw fields then parses them back. The class-based version writes labeled lines and reads/prints each full line without parsing the labels.

Q8. Why close a file?

Easy answer. Closing flushes buffered output and releases the file resource; stream destructors also close open files automatically.

Q9. What is a common data-validation omission?

Easy answer. The programs mostly check open success but do not verify every extraction/write operation succeeded.

Q10. what is RAII for file streams? [General knowledge]

Easy answer. A stream object closes its file automatically when the object is destroyed.

Experiment 8 - Text File Handling with Streams - expanded answers

Practice question 1. Explain the direct text-file program.

Suggested spoken answer. It reads a student’s name, roll number, and marks, writes them as text lines to student.txt, closes the output stream, then opens the same file with ifstream. It reads the fields back and prints them. The program checks whether each open succeeded.

Practice question 2. Why does the class version call cin.ignore?

Suggested spoken answer. The name is read with getline, but roll number and grade use >>, which leave the newline in the input buffer. ignore discards that leftover newline so the following getline receives the address rather than an empty string.

Practice question 3. Compare the two text formats.

Suggested spoken answer. The first program stores one field per line and knows the exact order when reading it. The class-based version writes labels such as Student Name and then displays lines as text. It does not parse the labels back into fields; it simply reads and prints each whole line.

Experiment 8B - Binary File I/O

Program focus: A Student struct with fixed character arrays is written/read as raw bytes using write/read and ios::binary.

Quick viva questions

Q1. What is binary file I/O?

Easy answer. It reads or writes bytes directly rather than converting values to a human-readable text representation.

Q2. What does outfile.write((char*)&s,sizeof(s)) do?

Easy answer. It writes sizeof(s) bytes beginning at the address of s.

Q3. Why is ios::binary used?

Easy answer. It requests binary mode and avoids text-mode transformations on platforms where those apply.

Q4. What does infile.read((char*)&s1,sizeof(s1)) do?

Easy answer. It reads that many bytes into the memory occupied by s1.

Q5. Why use setw(sizeof(s.name)) before reading a char array?

Easy answer. The width bounds the character extraction; width includes room for the terminating null, preventing overflow when used correctly.

Q6. Can raw struct files be safely exchanged across all machines?

Easy answer. No. Padding, byte order, type sizes, and representation can differ between compilers/platforms.

Q7. What input limitation comes from cin >> s.name?

Easy answer. It stops at whitespace, so names with spaces are not accepted as one field.

Q8. What error checks are missing?

Easy answer. The program does not check whether the binary files opened or whether write/read completed the expected number of bytes.

Q9. Compare text and binary files.

Easy answer. Text is human-readable and easier to inspect; raw binary can be compact/simple locally but is less portable.

Q10. why serialize fields instead of raw struct bytes? [General knowledge]

Easy answer. Explicit serialization defines a stable format independent of padding and machine representation.

Experiment 8B - Binary File I/O - expanded answers

Practice question 1. Explain how the binary program saves and restores a Student.

Suggested spoken answer. It collects values into a struct, opens student.dat in binary mode, and writes the raw bytes of the struct. It then reads the same number of bytes into another Student object and prints those fields.

Practice question 2. What are the main portability limitations?

Suggested spoken answer. A struct can contain padding bytes, and machines can differ in integer size, byte order, and floating-point representation. Therefore this raw file is suitable only for compatible environments; a portable file format should encode each field explicitly.

Practice question 3. How does setw protect the fixed character array?

Suggested spoken answer. For extraction into a char array, the width limits the characters read and must leave space for the null terminator. The program sets the width to the array size before reading. Formatted extraction also stops at whitespace, so a multiword name needs a different input method.

Experiment 9 - Function Templates and Bubble Sort

Program focus: Two template programs bubble-sort int, float, char, string, and Date arrays. Date defines operator> and operator<<.

Quick viva questions

Q1. What is a function template?

Easy answer. A reusable function pattern from which the compiler can generate type-specific functions when used.

Q2. What operations does bubbleSort require from T?

Easy answer. It must support comparison with > and assignment/copying so adjacent elements can be swapped.

Q3. What is the time complexity?

Easy answer. The nested loops take O(n^2) comparisons and swaps in the worst case; this version has no early-exit optimization.

Q4. Is this bubble sort stable?

Easy answer. Yes for equal values, because it swaps only when left > right, not when equal.

Q5. How are strings sorted?

Easy answer. The built-in string comparison provides lexicographic ordering.

Q6. Why does Date implement operator>?

Easy answer. The generic sorting template uses > to decide whether adjacent dates are out of order.

Q7. Explain the Date comparison order.

Easy answer. It compares year first, then month if years match, then day if both year and month match.

Q8. Why overload operator<< for Date?

Easy answer. The display template prints elements using <<, so Date needs an output-stream overload.

Q9. What should operator<< return?

Easy answer. ostream& so output can be chained and the stream can be returned to the caller.

Q10. when does template instantiation occur? [General knowledge]

Easy answer. The compiler generates a specialization when it sees a use with a particular type, subject to required operations being available.

Experiment 9 - Function Templates and Bubble Sort - expanded answers

Practice question 1. Explain how the template sorting code works for different types.

Suggested spoken answer. The compiler creates a version of bubbleSort for each type used, such as int, float, char, string, or Date. The algorithm compares neighbors with > and swaps them using a temporary T value. A type must support those operations for the template to compile.

Practice question 2. How does Date get sorted chronologically?

Suggested spoken answer. Date overloads >. It compares years first; if those match, it compares months; if those also match, it compares days. That creates chronological ordering. The separate << overload allows the display template to print each Date.

Practice question 3. Give complexity and stability.

Suggested spoken answer. The nested loops make O(n^2) comparisons in the worst case and use O(1) extra storage. The algorithm is stable because equal elements are not swapped: the condition is strictly greater-than.

Experiment 10 - Exception Handling

Program focus: One program throws an int for denominator zero; another throws int, char, double, or a string literal and uses typed catches plus catch(...).

Quick viva questions

Q1. What is an exception?

Easy answer. A mechanism to signal an error or unusual condition and transfer control to a matching handler.

Q2. What does throw do?

Easy answer. It raises an exception and searches outward for a compatible catch block.

Q3. How does ex10a prevent division by zero?

Easy answer. It checks b==0 and throws b before performing integer division.

Q4. Why is the catch parameter int?

Easy answer. The thrown expression b has type int, so catch(int) matches it.

Q5. How are multiple catch blocks selected?

Easy answer. The first compatible handler in order is selected.

Q6. What does catch(...) catch?

Easy answer. Any exception type not matched by earlier handlers; it should generally come last.

Q7. What happens for choice 4 in ex10b?

Easy answer. It throws the string literal "Invalid choice", whose type is const char*, so none of the int/char/double handlers match and catch(...) handles it.

Q8. Why is division by zero dangerous in C++?

Easy answer. Integer division by zero has undefined behavior; it must be checked before the division.

Q9. Does printing Program executed successfully mean no exception occurred?

Easy answer. Not necessarily. It prints after the try/catch, including after a handled exception.

Q10. why catch standard exceptions by const reference? [General knowledge]

Easy answer. It avoids copying and preserves polymorphic exception information.

Experiment 10 - Exception Handling - expanded answers

Practice question 1. Explain try, throw, and catch using the division program.

Suggested spoken answer. The program reads a numerator and denominator. Inside try it checks whether the denominator is zero; if so, it throws an int before division. The catch(int) block handles that type, prints an error, and then the program continues after the handler.

Practice question 2. Explain how the multiple catches work.

Suggested spoken answer. The program throws values of different types depending on the choice. C++ checks the catch blocks in order and executes the first compatible one. For an invalid choice, a string literal is thrown; that does not match int, char, or double, so the catch-all handler runs.

Practice question 3. What is the difference between catching an error and preventing it?

Suggested spoken answer. The zero-denominator check prevents the invalid division from being evaluated by detecting it first. The catch block handles the signal and lets the program continue in a controlled way. Both parts are needed for this example's behavior.

PDF Program 4(C) - Multiple Inheritance and Construction Order

Program focus: Alpha and Beta each have constructors; Gamma inherits publicly from both and initializes them in its member-initializer list.

Quick viva questions

Q1. What is multiple inheritance?

Easy answer. A class derives directly from more than one base class.

Q2. How is Gamma declared?

Easy answer. Gamma publicly inherits from Alpha and Beta, so it contains both base-class subobjects.

Q3. What is a member-initializer list?

Easy answer. The colon section after a constructor signature that initializes base classes and members before the constructor body executes.

Q4. What is the constructor order here?

Easy answer. Base classes initialize in the order they are declared: Alpha, then Beta; then Gamma's body runs.

Q5. Does initializer-list text order decide base order?

Easy answer. No. Base declaration order controls construction order.

Q6. Why initialize Alpha(x) and Beta(y)?

Easy answer. Their constructors require values, so Gamma forwards the matching arguments.

Q7. What is the output sequence conceptually?

Easy answer. Alpha constructor message and value, then Beta message and value, then Gamma message and value.

Q8. What is a possible multiple-inheritance problem?

Easy answer. Two bases may provide same-named members, causing ambiguity unless qualified or redesigned.

Q9. what is a base subobject? [General knowledge]

Easy answer. The portion of a derived object that represents one of its base-class instances.

PDF Program 4(C) - Multiple Inheritance and Construction Order - expanded answers

Practice question 1. Explain Gamma construction.

Suggested spoken answer. Creating Gamma g(10,20,30) first constructs its base subobjects. Alpha receives 10 and Beta receives 20 through the initializer list. After both base constructors finish, Gamma's constructor body stores 30 and prints its own message.

Practice question 2. What determines the constructor order?

Suggested spoken answer. The order of base classes in the class declaration determines construction order, not the order of entries in the initializer list. Here Alpha is listed before Beta, so Alpha is constructed first.

Practice question 3. What does multiple inheritance provide and complicate?

Suggested spoken answer. It lets one class combine behavior or data from multiple bases. It can also create ambiguous inherited names and diamond-shaped inheritance, which may need qualification or virtual inheritance.

PDF Program 5(A) - Hybrid and Virtual Inheritance

Program focus: Student and Teacher virtually inherit Person; GradTeachingFellow inherits from both and combines their data and display methods.

Quick viva questions

Q1. What is hybrid inheritance in this example?

Easy answer. It combines multiple inheritance and hierarchical inheritance: Student and Teacher are separate derived paths from Person, and GradTeachingFellow derives from both.

Q2. Why make Person a virtual base?

Easy answer. It ensures the final GradTeachingFellow contains one shared Person subobject instead of duplicate Person subobjects through Student and Teacher.

Q3. Which fields are protected and why?

Easy answer. Person fields and Student/Teacher fields are protected, allowing derived classes to use them while keeping them inaccessible to unrelated code.

Q4. Who initializes a virtual base?

Easy answer. The most-derived class is responsible for initializing a virtual base. Person has an implicit default constructor in this example, so no explicit initializer is needed.

Q5. What ambiguity would occur without virtual inheritance?

Easy answer. The final class would contain two Person subobjects and could have ambiguous access to members such as mName.

Q6. Why do getName and getAddress use cin>>?

Easy answer. Formatted extraction reads one whitespace-delimited token; a multiword name or address would be truncated.

Q7. Which display methods run in g.display()?

Easy answer. It calls the shared Person display, Student display, Teacher display, then prints the supervisor.

Q8. what if Person had only a parameterized constructor? [General knowledge]

Easy answer. GradTeachingFellow, as the most-derived class, would need to initialize the virtual Person base explicitly.

PDF Program 5(A) - Hybrid and Virtual Inheritance - expanded answers

Practice question 1. Explain why virtual inheritance is used.

Suggested spoken answer. Both Student and Teacher inherit from Person, and GradTeachingFellow inherits from both. Without virtual inheritance, the final object would contain two Person subobjects. Declaring Person as a virtual base makes the two paths share one Person part.

Practice question 2. Explain the data collected by the final class.

Suggested spoken answer. GradTeachingFellow combines Person information such as name and birth date, Student information such as address and department, Teacher information such as rank and course, and its own supervisor field. Its display function calls the relevant base-class display functions.

Practice question 3. Who is responsible for constructing the virtual base?

Suggested spoken answer. The most-derived class, here GradTeachingFellow, is responsible for initializing Person as a virtual base. In this code Person can be default-constructed, so an explicit Person initializer is unnecessary; if Person required arguments, the final class would supply them.

PDF Programs 5(B), 6(A), 6(B) - Runtime Polymorphism

Program focus: Account, Payment, and Media hierarchies use virtual functions and base pointers to invoke derived behavior.

Quick viva questions

Q1. What is runtime polymorphism?

Easy answer. A call through a base pointer/reference to a virtual function selects the override belonging to the object's actual derived type.

Q2. What does override communicate?

Easy answer. It asks the compiler to verify that a derived member overrides a virtual base member.

Q3. What interests are calculated?

Easy answer. SavingAccount returns 6% of balance; SalaryAccount 5%; CurrentAccount 0, as simple program formulas.

Q4. What happens when Payment* points to a CreditCard?

Easy answer. A call to virtual makepayment dispatches to CreditCard::makepayment.

Q5. Why is displayPayment nonvirtual?

Easy answer. It prints shared base fields and has no derived override in these examples.

Q6. Why do base classes have virtual destructors?

Easy answer. They make deletion through a base pointer safe by ensuring the derived destructor is called first.

Q7. How does Media::display behave through Media*?

Easy answer. Because display is virtual, pointing to a Book calls Book::display and pointing to a Tape calls Tape::display.

Q8. Are issue() and deposit() virtual?

Easy answer. No, but they are inherited nonvirtual functions and can still be called through Media*; they update the common issued field.

Q9. What is the difference between overloading and overriding?

Easy answer. Overloading uses same name with different parameter lists; overriding replaces a virtual base function with matching signature in a derived class.

Q10. what is a pure virtual function? [General knowledge]

Easy answer. A virtual function declared =0 that makes a class abstract until derived classes provide an implementation.

PDF Programs 5(B), 6(A), 6(B) - Runtime Polymorphism - expanded answers

Practice question 1. Explain runtime dispatch with the Account example.

Suggested spoken answer. Account has a virtual calculateInterest function. A base pointer is assigned to a SavingAccount, CurrentAccount, and SalaryAccount in turn. Each call selects the matching derived implementation, producing 6%, 0%, and 5% of the stored balance.

Practice question 2. Explain the Payment and Media examples.

Suggested spoken answer. Payment defines a virtual makepayment function that CreditCard, UPI, and Cash override. Media defines virtual display, and Book and Tape provide their own details. In both cases a base pointer can refer to different derived objects and the correct override runs at runtime.

Practice question 3. Why are virtual destructors important here?

Suggested spoken answer. The examples declare virtual destructors in polymorphic bases. If a derived object were allocated dynamically and then deleted through a base pointer, the virtual destructor ensures the derived destructor is called before the base destructor. The shown examples use stack objects, but the base class is safely prepared for polymorphic ownership.

Open-Ended 1 - Lambda with Reference Capture

Program focus: The lambda uses [&] to capture used external variables by reference, modifies a and b, and prints them before, inside, and after the call.

Quick viva questions

Q1. What is a lambda expression?

Easy answer. An inline callable object that can capture variables from its surrounding scope and execute a function body.

Q2. What does [&] mean?

Easy answer. Default-capture external variables by reference when used in the lambda.

Q3. What are the values before and after modify()?

Easy answer. Initially a=10,b=20. The lambda adds 5 and 10, so afterward a=15,b=30.

Q4. Why do the outer variables change?

Easy answer. The capture is by reference, so the lambda works with the original variables rather than copies.

Q5. What would [=] do instead?

Easy answer. It would capture used variables by value; modifying a captured copy would not modify the original variables.

Q6. What risk comes with reference capture?

Easy answer. The lambda must not outlive referenced variables, or it could hold dangling references.

Q7. What is auto used for?

Easy answer. It lets the compiler infer the lambda closure object's unique type.

Q8. what is a closure? [General knowledge]

Easy answer. The callable object created by a lambda, including its captured state.

Open-Ended 1 - Lambda with Reference Capture - expanded answers

Practice question 1. Explain the reference-capture lambda.

Suggested spoken answer. The program begins with a=10 and b=20. The lambda uses [&], which captures the external variables it uses by reference. It adds 5 to a and 10 to b, so the values printed after the call are 15 and 30.

Practice question 2. How would value capture differ?

Suggested spoken answer. With value capture, the lambda would receive copies of a and b. Changing those copies would not change the original variables in main. Reference capture directly modifies the originals.

Practice question 3. What lifetime caution applies?

Suggested spoken answer. A reference-capturing lambda should not be used after the referenced variables have gone out of scope. Otherwise its captured references would dangle.

Open-Ended 2 - std::vector Operations

Program focus: A vector<int> receives five values with push_back, is displayed with indexing and at(), adds 60, removes the last element, and reports sizes.

Quick viva questions

Q1. What is std::vector?

Easy answer. A dynamic-size sequence container that stores elements contiguously and manages its storage automatically.

Q2. What does push_back do?

Easy answer. It appends an element to the end and increases size by one.

Q3. What does pop_back do?

Easy answer. It removes the last element; it does not return that value.

Q4. What is size versus capacity?

Easy answer. Size is the number of live elements; capacity is how many can fit before another allocation is needed.

Q5. What is the difference between at() and []?

Easy answer. at() checks bounds and throws out_of_range on an invalid index; [] does not perform a bounds check.

Q6. What are the vector sizes in the program?

Easy answer. After five inserts size is 5; after push_back(60) it is 6; after pop_back it returns to 5.

Q7. What does the range-based for loop do?

Easy answer. It visits each element in the vector in sequence without manually managing an index.

Q8. What is the usual complexity of push_back?

Easy answer. Amortized O(1); an occasional reallocation can take O(n).

Q9. What happens if pop_back is called on an empty vector?

Easy answer. It violates the operation’s nonempty precondition; the caller should check empty() first.

Q10. what can reallocation invalidate? [General knowledge]

Easy answer. References, pointers, and iterators to vector elements may be invalidated when capacity changes.

Open-Ended 2 - std::vector Operations - expanded answers

Practice question 1. Explain the vector program step by step.

Suggested spoken answer. It starts with an empty vector and appends 10, 20, 30, 40, and 50. It prints the values and size, appends 60 so the size becomes six, then pop_back removes that last element and the final size returns to five.

Practice question 2. Compare at() and bracket indexing.

Suggested spoken answer. Both access an element by index. at() checks whether the index is within the vector and reports an out-of-range error if not; [] is unchecked and requires the programmer to use a valid index.

Practice question 3. Explain vector growth and complexity.

Suggested spoken answer. A vector manages a dynamic contiguous array. push_back is amortized constant time because most appends are cheap, although an occasional capacity increase copies or moves existing values and costs linear time. pop_back removes the last item in constant time when the vector is nonempty.

Rapid Revision

Topic	Remember
Bases	hex, oct, dec alter formatted output; base setting persists until changed.
Records	struct bundles fields; getline reads spaces; validate numeric ranges.
GCD/LCM	gcd is common divisor; lcm divide by gcd before multiply; overflow still possible.
OOP	class encapsulates data/behavior; overloading is compile-time choice; friend is non-member access.
Constructors	parameterized initializes from inputs; copy constructor creates new object from another.
Memory	new[] pairs with delete[]; vector/RAII manages lifetime.
Files	ifstream reads, ofstream writes; text readable; raw struct binary is platform-specific.
Templates	generic code needs operations supported by T; bubble sort O(n^2), stable here.
Exceptions	throw signals; typed catch handles; catch(...) last.
Inheritance	base construction order follows declaration; virtual base shares diamond base.
Virtual functions	base pointer calls derived override; base destructor virtual for polymorphic deletion.
Lambda	[&] captures used external variables by reference.
Vector	push_back amortized O(1); at bounds checks; pop_back requires nonempty.

Mock Viva

Answer aloud before reading each checkpoint.

1. For 26, what does the number conversion program print?

Checkpoint. 0x1a, 032, and 26 with base prefixes enabled.

2. What is the LCM of 12 and 18?

Checkpoint. 36; their GCD is 6.

3. Which area is calculated for a sphere?

Checkpoint. Surface area, 4*pi*r^2, not volume.

4. What is the difference between copy construction and assignment?

Checkpoint. One creates a new object; the other updates an existing object.

5. Why must new[] pair with delete[]?

Checkpoint. The allocation and deallocation forms must match.

6. Why call cin.ignore before reading an address?

Checkpoint. To remove the newline left behind by numeric extraction.

7. Why is raw binary struct storage not portable?

Checkpoint. Padding and field representations can differ across systems.

8. What operations must Date provide for the sorting template?

Checkpoint. Comparison with >, assignment/swapping, and output with << for display.

9. What catches an invalid-choice string literal in ex10b?

Checkpoint. catch(...) because it does not match int, char, or double.

10. What problem does virtual inheritance solve in the GTF class?

Checkpoint. It ensures Student and Teacher share one Person base subobject.

11. Why use virtual functions in the account/payment/media examples?

Checkpoint. To call the derived implementation through a base pointer.

12. What changes after calling the reference-capture lambda?

Checkpoint. a becomes 15 and b becomes 30.

13. What size does the vector have after push_back(60) and pop_back()?

Checkpoint. It ends at size 5.

Oral answer checklist

Define the concept; explain the exact code path; trace a sample; mention boundary conditions or a source limitation; state complexity where it applies; compare with a related C++ feature.

References and source notes

Primary source: supplied pcc.md, which compiles 20 source files and five additional PDF-only programs into 25 unique program listings. The source notes identify code corrections and state that code was not compiled or run.

std::gcd reference
https://en.cppreference.com/w/cpp/numeric/gcd

C++ virtual inheritance reference
https://en.cppreference.com/w/cpp/language/derived_class

C++ virtual function and destructor reference
https://en.cppreference.com/w/cpp/language/virtual

C++ vector push_back reference
https://en.cppreference.com/w/cpp/container/vector/push_back

C++ vector pop_back reference
https://en.cppreference.com/w/cpp/container/vector/pop_back

General-knowledge extensions are labeled. C++ standard/library requirements are stated where relevant, including C++17 for std::gcd.